Stages of Project Implementation

Individual Project №1

Name David Michael Francis
Discipline Operating Systems
University RUDN University, Moscow, Russia
Language English

Purpose of the work

Learn how to host a site on a GitHub page and complete the first stage of a personal individual project.

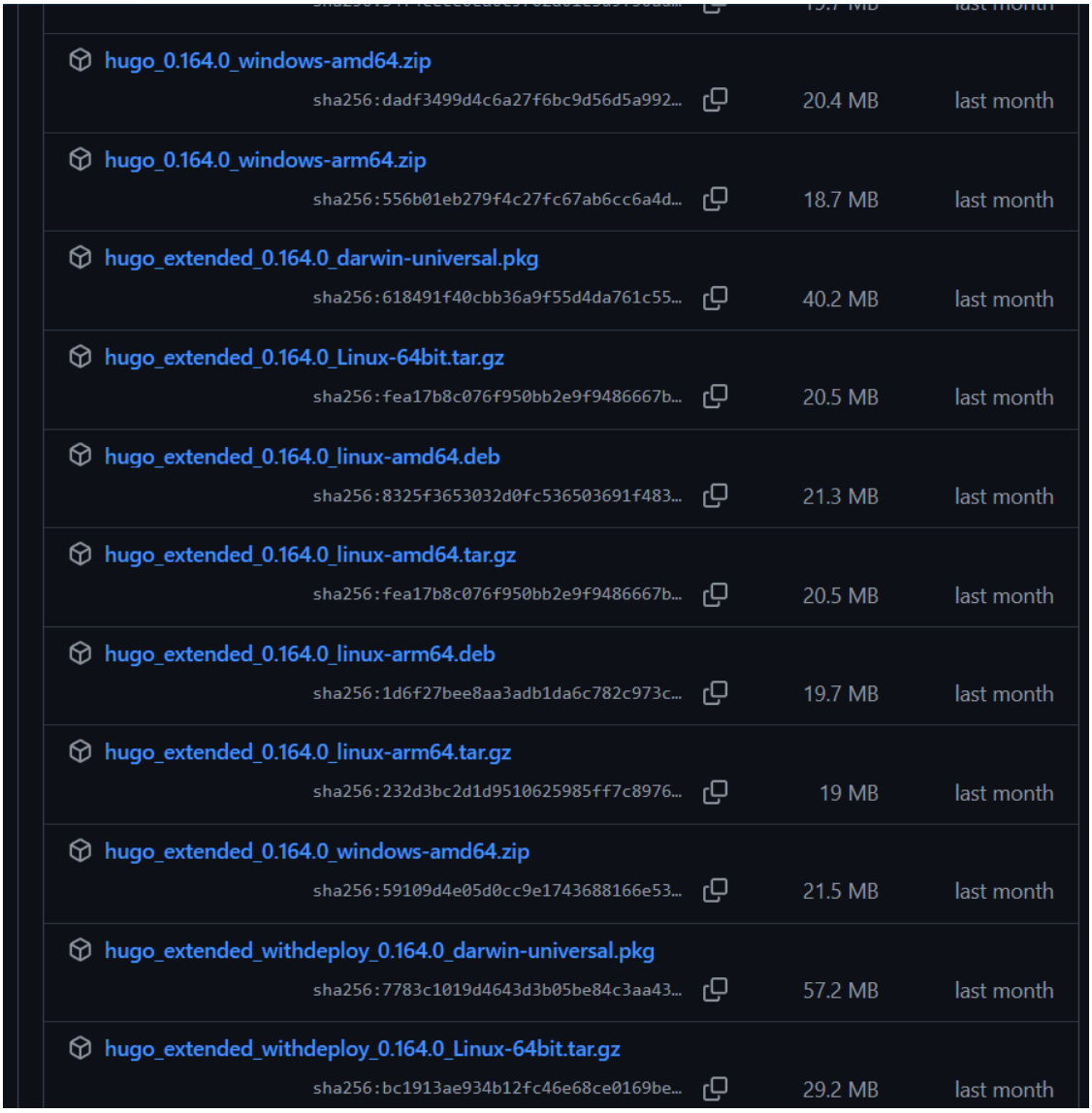
Task

1. Install the necessary software
2. Download Website Theme Template
3. Host it on git hosting
4. Set the parameter for site URLs
5. Place the site template on GitHub pages

---

1. Install the necessary software

The first step was installing Hugo, the static site generator used to build the website. I downloaded the extended edition (which supports Sass/SCSS) from the official Hugo releases page on GitHub.



screenshot of the Hugo releases page on GitHub, showing the `hugo_extended_0.164.0_linux-amd64.deb` asset

The file was downloaded to my Downloads folder inside WSL Ubuntu.

```
(base) mike@DESKTOP-I4EKTCO:~/downloads$ ls
Template-MMG-tex.zip
Template-MMG-tex.zip:Zone.Identifier
ettercap-0.8.3.1
ettercap-0.8.3.1.tar.gz
ettercap-0.8.3.1.tar.gz:Zone.Identifier
hugo_extended_0.164.0_linux-amd64.deb
hugo_extended_0.164.0_linux-amd64.deb:Zone.Identifier
markov-chain.log
markov-chain.tex
```

terminal output of `ls` showing the downloaded `.deb` file

I installed it using apt:

```
sudo apt install ./hugo_extended_0.164.0_linux-amd64.deb
```

Once installed, I confirmed the version and that the extended build was active:

```
hugo version
```

```
(base) mike@DESKTOP-I4EKTCO:/tmp$ hugo version
hugo v0.164.0-ce2470e7012b5ab5fc4e10ebe4027e9f8d9e00dc+extended linux/amd64 BuildDate=2026-07-06T16:39:30Z Ven
dorInfo=gohugoio
```

terminal output showing hugo v0.164.0+extended

I also confirmed Git was installed and configured my identity for commits:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

---

2. Download Website Theme Template

Next, I created a new Hugo project and initialised it as a Git repository.

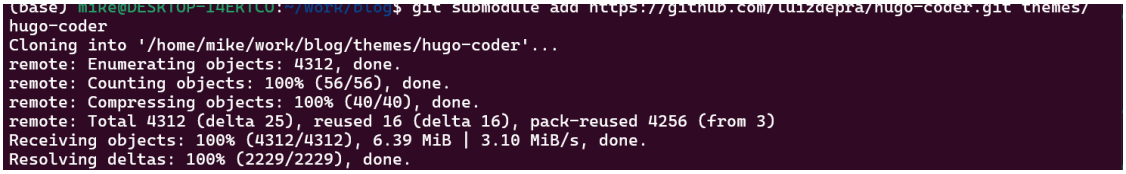
```
hugo new site blog
cd blog
git init
```

```
(base) mike@DESKTOP-I4EKTCO:~/work$ hugo new site blog
Congratulations! Your new Hugo project was created in /home/mike/work/blog.

Just a few more steps...

1. Change the current directory to /home/mike/work/blog.
2. Create or install a theme:
   - Create a new theme with the command "hugo new theme <THEMENAME>"
   - Or, install a theme from https://themes.gohugo.io/
3. Edit hugo.toml, setting the "theme" property to the theme name.
4. Create new content with the command "hugo new content <SECTIONNAME>/<FILENAME>.<FORMAT>".
5. Start the embedded web server with the command "hugo server --buildDrafts".

See documentation at https://gohugo.io/.
(base) mike@DESKTOP-I4EKTCO:~/work$ cd blog
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/mike/work/blog/.git/
```

terminal output showing the site scaffold being created

I then chose a theme from the Hugo themes directory and added it as a Git submodule of the project:

```
git submodule add https://github.com/luizdepra/hugo-coder.git themes/hugo-coder
```



terminal output confirming the theme was cloned successfully

Adding the theme as a submodule keeps it version-controlled separately from the main project, which is the standard way Hugo themes are managed.

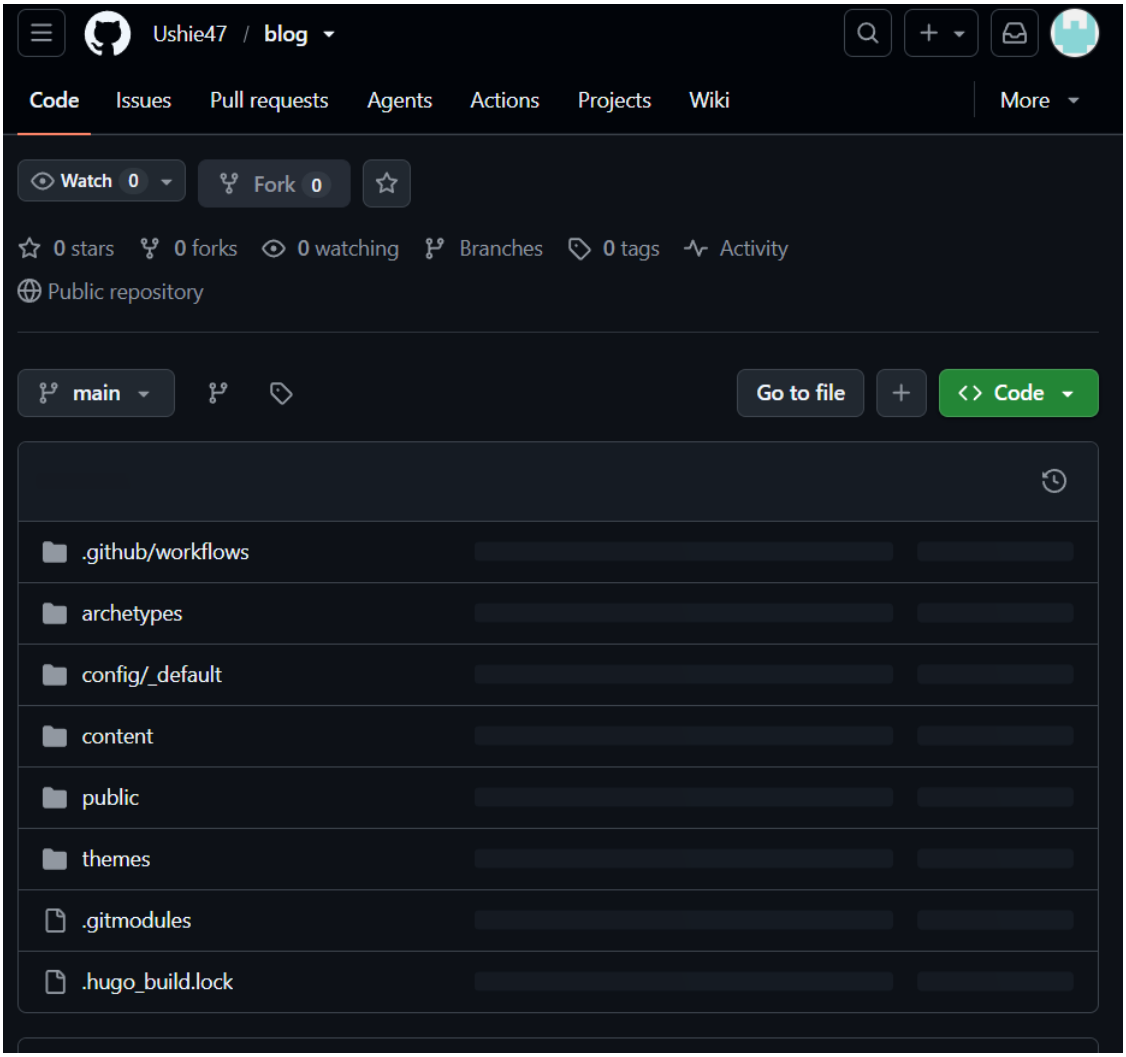
---

3. Host it on git hosting

Before pushing the project online, I created a personal access setup for authentication and previewed the site locally to confirm it built correctly:

```
hugo server
```

Once confirmed locally, I created a new empty repository on GitHub to host the project.



"Create a new repository" page on GitHub

I linked my local project to the new GitHub repository and pushed the code:

```
git add .
git commit -m "Initial Hugo site"
git remote add origin git@github.com:Ushie47/blog.git
git branch -M main
git push -u origin main
```

```
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 683 bytes | 136.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:Ushie47/blog.git
 * [new branch]      main -> main
```

terminal output confirming the push succeeded

4. Set the parameter for site URLs

For the site to render correctly once deployed, the `baseURL` parameter in the Hugo configuration file needed to match the exact URL where GitHub Pages would serve the site.

```
nano hugo.toml
```

```
baseURL = "https://ushie47.github.io/blog/"
```

I committed and pushed this change:

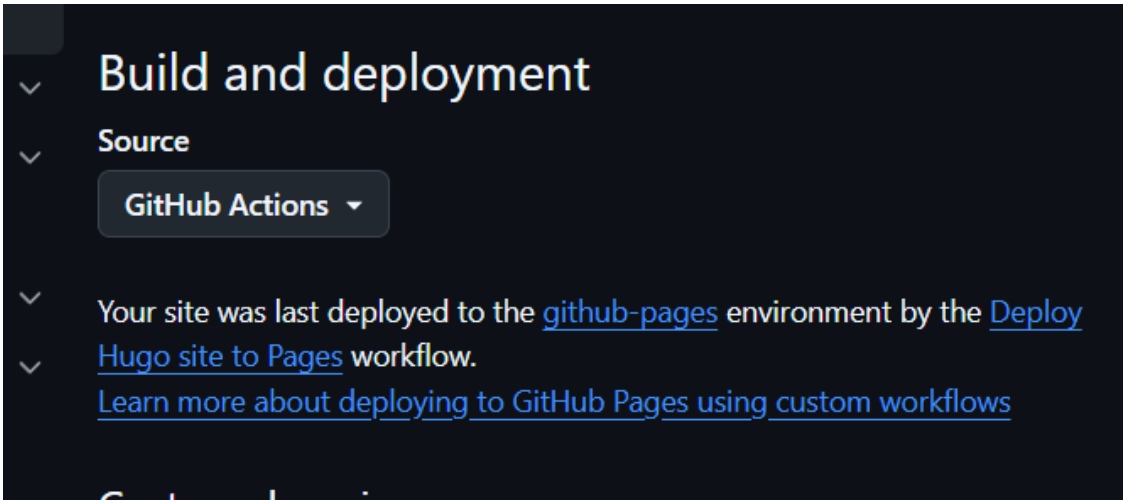
```
git add hugo.toml
git commit -m "Set baseURL and site config"
git push
```

```
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git add hugo.toml
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git commit -m "Set baseURL"
[main 0f71bd8] Set baseURL
 1 file changed, 24 insertions(+), 3 deletions(-)
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 545 bytes | 545.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Ushie47/blog.git
   6f9fe3b..0f71bd8  main -> main
```

terminal output confirming the push

5. Place the site template on GitHub pages

To deploy the site automatically, I enabled GitHub Pages with the “GitHub Actions” build source, under repository **Settings → Pages**.



screenshot of the GitHub Pages settings, with Source set to "GitHub Actions"

I then created a deployment workflow file to build and publish the site on every push to the main branch:

```
mkdir -p .github/workflows
nano .github/workflows/hugo.yml
```

```
git add .github/workflows/hugo.yml
git commit -m "Add GitHub Pages deploy workflow"
git push
```

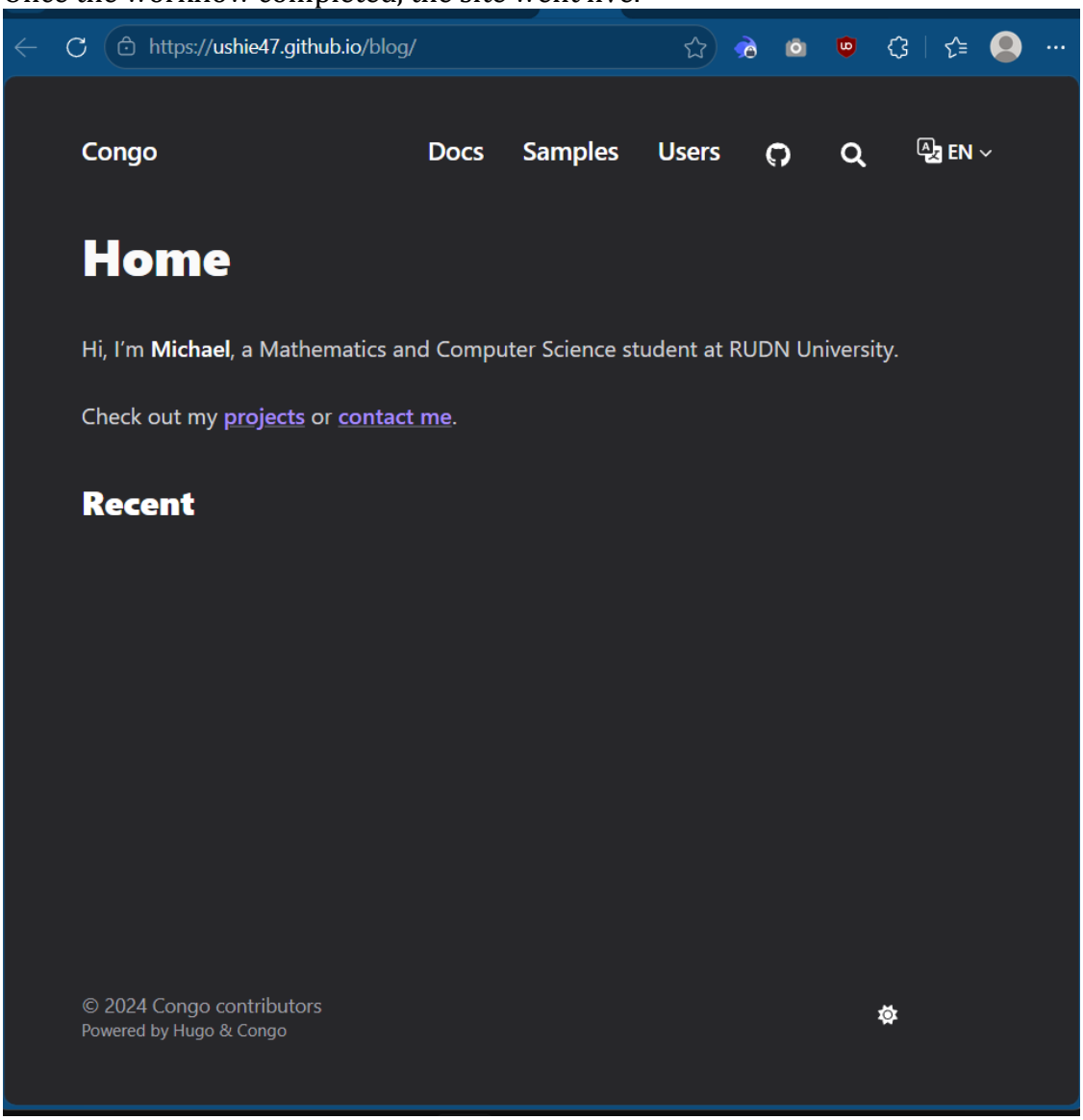
```
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ mkdir -p .github/workflows
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ nano .github/workflows/hugo.yml
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git add .github/workflows/hugo.yml
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git commit -m "Add github Pages"
[main e129ae4] Add github Pages
 1 file changed, 53 insertions(+)
 create mode 100644 .github/workflows/hugo.yml
(base) mike@DESKTOP-I4EKTCO:~/work/blog$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 837 bytes | 209.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Ushie47/blog.git
   0f71bd8..e129ae4  main -> main
```

terminal output confirming the push

I monitored the deployment under the repository's **Actions** tab.

screenshot of the GitHub Actions run completing successfully

Once the workflow completed, the site went live.



Conclusion

Through this stage, I learned how to install Hugo, structure a static site project, manage a theme as a Git submodule, configure a project for deployment, and automatically publish a website using GitHub Actions and GitHub Pages. This provided a working foundation for the personal website that will be expanded in later stages of the project.